



Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Compiladores - Prova 3 (20 pts)
Professor: Pedro Ramos

Estudante: _____

1. Otimização de código (6pts)

```
1 b := 2
2 i := 1
3 do {
4     a := b + 1
5     c := 2
6     if (i % 2) == 0 {
7         d := a + d
8         e := 1 + d
9     } else {
10        d := - c
11        f := 1 + a
12    }
13    i = i + 1
14 } while (a < 2)
```

(a) Aplique as seguintes otimizações ao programa acima, na ordem descrita a seguir. Mostre o programa resultante de cada otimização (3pts):

- i. Propagação de constantes
- ii. Eliminação de código morto

(b) Utilizando o código em questão como exemplo, e seus conhecimentos em Otimização de código, explique por que achar uma ordem ótima de aplicação de otimizações a um programa é NP-Difícil. (1pt)

(c) Indique pelo menos mais uma otimização além das listadas em (a).i e (a).ii que pode ser aplicada ao programa original antes das otimizações aplicadas em (a). A otimização deve produzir algum efeito, isto é, o programa resultante da otimização deve ser diferente do original em algum ponto. Mostre o efeito da otimização escolhida. (2pts)

2. **Alocação de registradores (6pts)** Para cada uma das funções abaixo, desenhe:

(a) O grafo de fluxo de controle (CFG) no nível de instrução.

(b) O grafo de interferência de registradores (variáveis temporárias). Variáveis vivas no mesmo ponto do código representam nós adjacentes no grafo de interferência de registradores (RIG).

FUNÇÃO I: (2pts)

```
1 f(a,b) {  
2     c := b;  
3     d := c;  
4     e := d;  
5     f := e;  
6     g := f;  
7     a := a + g;  
8  
9     return a;  
10 }
```

FUNÇÃO II: (4pts)

```
1 g(a) {  
2     if a then goto L1 else goto L2  
3  
4 L1:  
5     c := a;  
6     a := a + c;  
7     d := a;  
8     d := d + c;  
9     goto L3  
10  
11 L2:  
12     b := a;  
13     a := a + b;  
14     d := a;  
15     d := d + b;  
16  
17 L3:  
18     return d  
19 }
```

3. **Coletor de Lixo (6pts)** Considere um compilador que implementa as técnicas *Stop & Copy* (I) e *Mark & Sweep* (II) para coleta de lixo. Considere também que a coleta de lixo é **executada somente quando a memória do programa acaba**, isto é, quando uma chamada a *new* não pode ser atendida.

(a) Qual dos dois algoritmos ((I) e (II)) é mais rápido? Justifique. (2pts)

(b) Qual será executado com mais frequência? Justifique. (1pt)

(c) Python utiliza a técnica *Contagem de Referências* (III) para coleta de lixo. O *runtime* implementa um “detector de ciclos” para identificar referências cíclicas. Discuta como você implementaria um detector de ciclos para gerenciamento de memória automático. Quando é seguro **limpar um ciclo de objetos** da memória? (3pts)

4. **Closures (2pts)**. Linguagens de programação implementam *closures*, funções que vivem na memória atreladas a um ambiente léxico. No programa abaixo, uma função **interna** é retornada por uma função **externa**. A função **interna** é então invocada na linha 11:

```
1 func externa() {  
2     let x = new Object()  
3     func interna() {  
4         console.log(x.toString())  
5     }  
6  
7     return interna  
8 }  
9  
10 let f = externa()  
11 f()
```

Explique como o suporte a *closures* impacta otimizações como alocação de registradores e gerenciamento automático de memória (coletor de lixo).

5. **Estruturas do compilador (2pts)** Considere uma linguagem J que possui:

- Sintaxe semelhante à do Java.
- Definições aninhadas de funções dentro de outras funções.
- Variáveis locais (com endereços estáticos).
- Lançamento e tratamento de exceções nomeadas e sem parâmetros.

Qual dos seguintes conjuntos de estruturas de dados de tempo de execução é **mais provável de ser usado por um compilador** para a linguagem J para gerenciar suas funcionalidades? **Marque uma alternativa.**

- (A) Heap, Tabela Hash, Buffer Circular, Árvore de Busca Binária
- (B) Pilha de Ativação, Tabela de Exceções, Heap
- (C) Pilha de Ativação, Lista Encadeada, Heap, Trie
- (D) Array, Buffer Circular, Fila de Prioridade, Tabela de Exceções